

ChronoForecast — Unified Time-Series Benchmarking Framework

■■ ENDGAME TIER — ML Systems + Research Engineering

"Benchmark Classical, ML & Deep Learning Forecasters Across Datasets with a Live Leaderboard"

■■ Endgame Project | Domain: Time-Series ML + Deep Learning | Difficulty: Advanced | Timeline: 11 Weeks

■ Project Overview

Description	A rigorous, extensible benchmarking framework that evaluates classical (ARIMA, SARIMA, Exponential Smoothing), ML (XGBoost, LightGBM, Random Forest), and modern deep learning (PatchTST, TimesNet, N-HiTS) forecasting models across multiple real-world time-series datasets. Features automatic feature engineering, dataset-agnostic evaluation, a configurable leaderboard UI, and a statistical significance testing module to determine if performance differences are real.
Target Audience	ML researchers, time-series practitioners, senior ML engineers, academic competition builders
Domain	Time-Series Forecasting, Benchmarking, Deep Learning, Statistical Testing
Deliverable	Modular benchmarking framework + live leaderboard Streamlit app + research-grade evaluation report
Why It Stands Out	Rather than building one forecasting model, ChronoForecast builds the system used to evaluate many — demonstrating systems thinking, research methodology, and deep time-series expertise that very few ML portfolios show.

■ Tech Stack

Language	Python 3.10+
Classical Models	statsmodels (ARIMA, SARIMA, Holt-Winters, ETS)
ML Models	XGBoost, LightGBM, scikit-learn (Random Forest, Ridge)
Deep Learning	PyTorch (PatchTST, N-HiTS via neuralforecast), TimesNet
DL Framework	Nixtla neuralforecast library
Auto Feature Eng.	tsfresh, tsfel for automatic time-series feature extraction
Evaluation	Custom RMSE, MAE, MAPE, SMAPE, CRPS; scipy for Diebold-Mariano test
Experiment Tracking	MLflow (log all runs, params, metrics, artifacts)
Dashboard	Streamlit (leaderboard + forecast visualisation)
Storage	SQLite or PostgreSQL (benchmark results DB)
Dev Tools	Jupyter, VSCode, Git, Docker

■ MVP Features (Phase 1)

Multi-Dataset Benchmark Suite

- ◆ Dataset 1: M4 Competition (monthly, quarterly, yearly subsets)
- ◆ Dataset 2: ETTh1/ETTh2 (electricity transformer temperature — standard DL benchmark)
- ◆ Dataset 3: Air Passengers (seasonal SARIMA benchmark)
- ◆ Dataset 4: Store Sales (Kaggle retail forecasting — multi-series with covariates)
- ◆ Dataset 5: Custom financial series (S&P; 500 closing prices — walk-forward setup)
- ◆ Unified DataLoader interface: all datasets load into a standard schema (timestamp, value, series_id)

Automatic Feature Engineering Pipeline

- ◆ Calendar features: hour, day, week, month, quarter, is_holiday, is_weekend
- ◆ Lag features: configurable lag set (t-1, t-7, t-14, t-28 for daily; t-1 to t-12 for monthly)
- ◆ Rolling statistics: 7/14/30-day rolling mean, std, min, max, median
- ◆ tsfresh: 63 statistical features per series (entropy, autocorrelation, peak counts)
- ◆ Fourier features: sine/cosine transforms of multiple seasonality periods

Model Registry

- ◆ Classical: ARIMA (auto-order via AIC), SARIMA, Holt-Winters, ETS, Theta Method
- ◆ ML: XGBoost, LightGBM, Random Forest, ElasticNet (with engineered features)
- ◆ Deep: PatchTST, N-HiTS, TimesNet via neuralforecast library
- ◆ Naive baselines: Last-value baseline, Seasonal naive, Moving average (for reference)
- ◆ Pluggable model interface: add any new model with a standard fit/predict API

Evaluation Engine

- ◆ Metrics: RMSE, MAE, MAPE, SMAPE, MASE, CRPS (probabilistic forecasts)
- ◆ Walk-forward validation for all models (expanding or sliding window)
- ◆ Diebold-Mariano statistical significance test: is Model A significantly better than Model B?
- ◆ Confidence intervals: bootstrap CI on all evaluation metrics
- ◆ Cross-dataset aggregated ranking: mean rank across all datasets (M4 competition-style)

Leaderboard & Visualisation

- ◆ Sortable leaderboard table: models as rows, datasets as columns, metrics as values
- ◆ Per-dataset forecast plots: actual vs predicted for top-5 models side by side
- ◆ Model rank progression chart: how each model ranks across dataset difficulty levels
- ◆ Statistical significance heatmap: pairwise DM test p-values for all model pairs

■ Development Roadmap

Week 1 Framework Architecture & Setup	■ Design modular framework architecture: datasets/, models/, evaluators/, features/, dashboard/ ■ Define standard interfaces: BaseDataset, BaseModel, BaseEvaluator abstract classes ■ Set up environment: install statsmodels, neuralforecast, tsfresh, MLflow ■ Set up MLflow tracking server (local) for logging all benchmark runs
Week 2 Dataset Loaders	■ Build M4 Dataset loader: download, parse, and split into train/test per frequency ■ Build ETTh1 loader: hourly electricity dataset, standard 96/192/336/720 prediction lengths ■ Build Air Passengers loader (baseline sanity check for seasonal models) ■ Implement unified DataLoader interface: all datasets return pandas DataFrame with standard schema
Week 3 Automatic Feature Engineering	■ Build calendar feature generator: hour, dayofweek, month, quarter, is_holiday ■ Build lag feature generator: configurable lag set per dataset frequency ■ Build rolling statistics generator: mean, std, min, max over multiple windows ■ Integrate tsfresh for automated statistical feature extraction (select top-50 features by relevance)
Week 4 Classical Model Implementations	■ Implement auto-ARIMA wrapper (pmdarima): auto-selects p,d,q orders via AIC ■ Implement SARIMA with seasonal period detection (FFT-based) ■ Implement Holt-Winters and ETS via statsmodels ■ Implement Theta Method (M3 competition winner for monthly series) ■ Log all classical model results to MLflow
Week 5 ML Model Implementations	■ Implement XGBoost and LightGBM forecasters with full feature engineering pipeline ■ Implement Random Forest and ElasticNet as additional baselines ■ Implement multi-step direct forecasting: one model per forecast horizon ■ Implement recursive forecasting: iterative single-step predictions ■ Compare direct vs recursive strategy on all ML models
Week 6 Deep Learning Model Integration	■ Install neuralforecast and set up GPU (or CPU fallback) environment ■ Integrate PatchTST: patch-based transformer for long-range forecasting ■ Integrate N-HiTS: hierarchical interpolation for multi-scale patterns ■ Integrate TimesNet: 2D temporal variation modelling ■ Log all DL training configs, epochs, and validation metrics to MLflow

Week 7 Walk-Forward Validation Engine	■ Implement expanding window walk-forward: grow training set, fixed test horizon ■ Implement sliding window walk-forward: fixed training window size ■ Run all models through walk-forward validation on all 5 datasets ■ Aggregate per-fold metrics into final evaluation summary per model-dataset pair
Week 8 Statistical Significance Testing	■ Implement Diebold-Mariano test: test if forecast error differences are statistically significant ■ Build pairwise significance matrix for all model pairs per dataset ■ Implement bootstrap confidence intervals on all evaluation metrics (1000 resamples) ■ Generate significance-annotated model comparison table
Week 9 Leaderboard & Streamlit Dashboard	■ Build SQLite results database: store all benchmark run results ■ Build Streamlit leaderboard page: sortable table with model rankings per dataset ■ Build forecast visualisation page: actual vs predicted for any model-dataset pair ■ Build statistical significance heatmap: pairwise DM test p-values ■ Add model rank progression radar chart across dataset difficulties
Week 10 Automated Report Generation	■ Build benchmark summary report generator (PDF via ReportLab) ■ Report sections: dataset descriptions, model configs, metric tables, significance tests, key findings ■ Generate 'Key Findings' section using template: which model wins on which dataset type and why ■ Validate report against published M4 and ETT benchmark papers for sanity
Week 11 Extensibility, Testing & Publish	■ Write plugin guide: how to add a new model or dataset to ChronoForecast in under 30 minutes ■ Write pytest suite for all core modules (data loaders, feature engineers, evaluators) ■ Stress test: run full benchmark suite end-to-end and measure total runtime ■ Publish on GitHub with clear README, benchmark results table, and reproducibility guide ■ Write a companion blog post or notebook explaining 3 key insights from the benchmark

■ Phase 2 — Future Enhancements

- ◆ Probabilistic forecasting: add conformal prediction and quantile regression for all models
 - ◆ Multivariate forecasting: extend all models to handle multiple correlated time series
 - ◆ Hierarchical forecasting: top-down and bottom-up reconciliation for retail hierarchies
 - ◆ AutoML for forecasting: meta-model that predicts which forecaster will win for a given dataset
 - ◆ Online benchmarking: continuously update leaderboard as new data arrives
 - ◆ Extend to 20+ datasets including traffic, weather, energy, and healthcare series
-

■ Potential Challenges & Solutions

Challenge: Model Training Time

Solution: DL models (PatchTST, TimesNet) take hours to train on large datasets — implement checkpoint saving, early stopping, and run DL models in a separate background process from the classical/ML suite

Challenge: Feature Leakage in Lag Features

Solution: Lag features must be carefully shifted: a lag-1 feature for period t must use the value at $t-1$ only; validate all lag features against the walk-forward split boundary to prevent leakage

Challenge: Metric Sensitivity to Scale

Solution: MAPE breaks down when actual values approach zero (division by zero) — always report SMAPE alongside MAPE and flag near-zero series; use MASE for series with intermittent zero demand

Challenge: Diebold-Mariano Test Assumptions

Solution: DM test assumes forecast errors are covariance stationary — verify this holds for each dataset; for long-horizon forecasts, use Harvey-Leybourne-Newbold correction to improve finite-sample performance

Challenge: Reproducibility

Solution: DL training is stochastic — fix random seeds for PyTorch, numpy, and Python, and log all seeds to MLflow; document hardware (CPU/GPU) used for benchmark to allow fair comparison

■ Learning Resources

- ◆ M4 Competition Dataset and results paper (Makridakis et al.)
- ◆ ETT Dataset paper: Informer (Zhou et al., 2021)

- ◆ Nixtla neuralforecast documentation — PatchTST, N-HiTS
 - ◆ statsmodels ARIMA, SARIMA documentation
 - ◆ Diebold-Mariano test — scipy.stats and statsmodels
 - ◆ tsfresh documentation — automated time-series feature extraction
 - ◆ MLflow tracking documentation
-

■ Project Completion Criteria

- Unified DataLoader interface working for all 5 datasets
- Automatic feature engineering pipeline with calendar, lag, rolling, and tsfresh features
- All classical models (ARIMA, SARIMA, Holt-Winters, ETS, Theta) implemented and validated
- All ML models (XGBoost, LightGBM, RF, ElasticNet) with walk-forward validation
- At least 2 DL models (PatchTST, N-HiTS) integrated via neuralforecast
- Walk-forward validation engine (expanding and sliding window) working for all models
- Diebold-Mariano significance tests computed for all model pairs
- All results logged to MLflow with full reproducibility
- Streamlit leaderboard with sortable rankings, forecast charts, and significance heatmap
- PDF benchmark report generated with key findings section
- GitHub published with README, results table, and plugin guide